

The Genealogy of Sound: Music Sampling Network Analysis

Spiegazione Completa del Progetto

Corso: Data Mining 2025/2026

Università: Università degli Studi di Trento

Studente: Jacopo Corrao (Matricola: 258412)

Professore: Giacomo Scettri

Indice

1. Panoramica del Progetto
 2. Introduzione – Sezione 1 del Report
 3. Preparazione dei Dati – Sezione 2 del Report
 4. Metodologia e Algoritmi – Sezione 3 del Report
 5. Risultati – Sezione 4 del Report
 6. Discussione e Interpretazione – Sezione 5 del Report
 7. Conclusione – Sezione 6 del Report
 8. Struttura del Progetto
-

1. Panoramica del Progetto

Di cosa si tratta?

Questo progetto analizza la **genealogia della musica** attraverso le relazioni di *sampling* (campionamento). Nella musica, fare sampling significa prendere un pezzo di una registrazione esistente e riutilizzarlo in una nuova canzone. È una pratica onnipresente nell'hip-hop, nell'elettronica, nel pop e in molti altri generi.

L'idea di base è modellare l'ecosistema musicale come un **grafo diretto** (un insieme di “nodi” collegati da “freccie”): - I **nodi** rappresentano le canzoni - Gli **archi** (freccie) rappresentano le relazioni di sampling: se la Canzone B campiona la Canzone A, disegniamo una freccia da B verso A

L'obiettivo è scoprire **chi sono i veri antenati strutturali della musica moderna** andando oltre le semplici classifiche di vendita o di streaming, usando tecniche di Data Mining come PageRank e rilevamento di comunità.

Le tre domande di ricerca

1. **Q1 – Chi sono gli antenati strutturali della musica moderna?**
Come possiamo andare oltre il semplice conteggio di quante volte un artista è stato campionato, per capire chi è veramente influente a livello strutturale?

2. **Q2 – Quali comunità esistono al di là delle etichette di genere?**
Possiamo scoprire “famiglie musicali” nascoste basandoci puramente su chi campiona chi, senza usare etichette come “hip-hop” o “rock”?
3. **Q3 – Come scorre l’influenza?** Gli algoritmi distribuiti su grafi possono rivelare percorsi genealogici inaspettati?

Risultati principali in sintesi

Metrica	Valore
Eventi di sampling	57.741
Canzoni uniche	67.638
Artisti unici	23.404
Comunità rilevate	1.774
Campionamenti intra-comunità	71,69%
Correlazione di Spearman	0,72

2. Introduzione – Sezione 1 del Report

Il problema

L’industria musicale misura tradizionalmente il successo attraverso **vendite e streaming**. Ma l’impatto culturale è strutturalmente diverso dalla popolarità. Una “one-hit wonder” può vendere milioni di copie, ma un oscuro brano funk degli anni ’70 potrebbe essere campionato da centinaia di artisti hip-hop, diventando la spina dorsale di un intero genere.

Esempio concreto: Prendiamo un brano come *Funky Drummer* di James Brown (1970). È stato campionato in migliaia di canzoni hip-hop, ma quante copie ha venduto rispetto a un successo pop dell’epoca? Pochissime. Eppure, la sua **influenza strutturale** è immensa. Il progetto vuole catturare proprio questo tipo di influenza.

La soluzione proposta

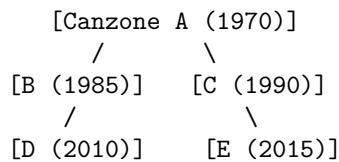
Modelliamo l’ecosistema musicale come un **grafo diretto** dove: - I **nodi** sono le canzoni - Gli **archi** rappresentano “ha campionato” (puntano dal campionatore all’originale)

Se la Canzone B (1985) campiona la Canzone A (1970), disegniamo:

B -> A

Il grafo concettuale (toy graph)

Per capire intuitivamente la topologia della rete, il report mostra un grafo concettuale semplificato (Figura 1 del report):



In questo esempio: - La Canzone A (1970) è la “fondatrice” della famiglia musicale: viene campionata da B e C - B e C vengono a loro volta campionate da D ed E - L’influenza scorre dal futuro al passato (da destra a sinistra, o dal basso verso l’alto)

PageRank permette di far scorrere l’“autorità” all’indietro lungo questi archi, identificando i nodi fondamentali (come la Canzone A). Gli algoritmi di community detection (Louvain) raggruppano i nodi strutturalmente correlati.

Perché è importante: Capire queste dinamiche strutturali è essenziale non solo per la musicologia, ma anche per costruire migliori sistemi di raccomandazione, gestire le questioni di copyright e comprendere come la cultura digitale si evolve attraverso il remix e il riutilizzo.

3. Preparazione dei Dati – Sezione 2 del Report

3.1 Fonte dei dati: MusicBrainz

I dati provengono da **MusicBrainz**, un’enciclopedia musicale aperta che raccoglie metadati contribuiti dagli utenti (licenza CC BY-NC-SA 3.0).

Perché MusicBrainz e non un dataset Kaggle?

MusicBrainz fornisce **dump grezzi del database PostgreSQL**. La complessità sta nel fatto che i dati sono in **Terza Forma Normale (3NF)** – cioè altamente normalizzati per evitare ridondanze, ma difficili da usare direttamente. Per ricostruire relazioni significative serve un significativo lavoro di data engineering.

3.2 I file raw (TSV senza intestazioni)

I dati grezzi sono file TSV (tab-separated values) senza intestazioni, estratti dall’archivio `mbdump.tar.bz2`. Ecco le tabelle principali:

Tabella	Cosa contiene
<code>recording</code>	Le registrazioni (canzoni) con ID e nomi
<code>artist</code>	Gli artisti con ID e nomi
<code>artist_credit_name</code>	Mappa i crediti degli artisti agli artisti specifici
<code>l_recording_recording</code>	Tabella ponte che collega registrazioni ad altre registrazioni (contiene TUTTE le relazioni inter-registrazione)
<code>link</code>	Metadati dei link (tipo di relazione)

Tabella	Cosa contiene
link_type	Descrizione testuale dei tipi di link
track, medium, release, release_group_meta	Usate per informazioni temporali (anni di pubblicazione)

3.3 Schema-on-Read

Poiché i file raw non hanno intestazioni, ho definito **manualmente lo schema di ogni tabella** basandomi sulla documentazione di MusicBrainz. Questo è più efficiente dell'inferenza automatica dello schema, che sarebbe costosa computazionalmente e soggetta a errori.

Esempio di schema manuale:

```
schema_recording = StructType([
    StructField("id", IntegerType(), nullable=False),
    StructField("gid", StringType(), nullable=True),
    StructField("name", StringType(), nullable=True),
    StructField("artist_credit", IntegerType(), nullable=True),
    # ... altri campi
])
```

3.4 Il percorso per ricostruire il grafo

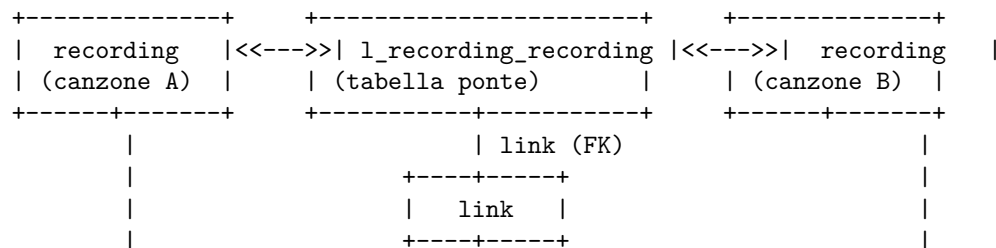
Per ricostruire “chi ha campionato chi”, dobbiamo navigare una struttura relazionale complessa. Ecco il percorso:

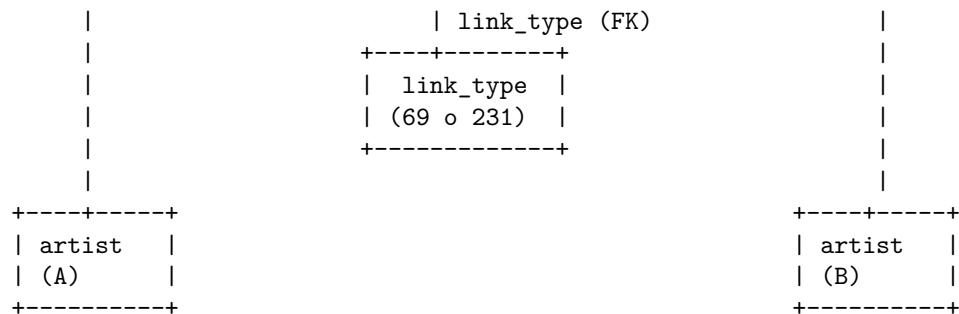
```
recording (canzone A)
  -> l_recording_recording (tabella ponte: entity0 = A, entity1 = B)
    -> recording (canzone B)
      -> artist_credit_name (chi ha fatto B?)
        -> artist (nome dell'artista)
```

Contemporaneamente:

```
l_recording_recording.link -> link.id -> link_type.id
  -> filtro per link_type = 69 o 231 (solo sampling)
```

Schema ER (semplificato):





3.5 Filtraggio: isolare solo il sampling

Un passaggio critico è stato isolare solo le relazioni di **sampling puro** dai milioni di interazioni generiche presenti in `l_recording_recording`. Questa tabella contiene infatti anche: - **Cover**: reinterpretazioni di una canzone - **Remix**: versioni alternative - **Medley**: mashup di più canzoni - **Registrazioni live**: esecuzioni dal vivo

Includere tutti questi tipi di link avrebbe creato un grafo rumoroso, dove l’“influenza” strutturale (il riutilizzo fisico di audio) si confonde con la “reinter-pretazione” artistica.

Per ottenere una genealogia precisa – definita come casi in cui un pezzo di audio è fisicamente riutilizzato – ho filtrato solo i Link Type ID specifici:

ID	Descrizione	Categoria
69	“Samples material”	Tipo legacy
231	“Is sampled by / Samples material”	Tipo corrente

Perché due ID? MusicBrainz ha due ID diversi per la stessa cosa perché nel tempo hanno cambiato lo schema dei tipi di link. Il 69 è il “vecchio” ID, il 231 è il “nuovo”. Entrambi rappresentano la stessa relazione.

3.6 Rimozione dei self-loop

Durante l’analisi esplorativa è emerso un problema critico di qualità dei dati: la presenza di **self-loop** – archi dove un artista campiona se stesso.

Quanti? 1.407 self-loop (circa il 6,4% di tutti gli archi).

Cosa rappresentano? - **Remix**: un artista che remixa la propria traccia - **Ripubblicazioni**: versioni diverse della stessa canzone (album vs singolo) - **Struttura dell’album**: tracce intro/outro che campionano altre canzoni dello stesso album - **Artefatti di dati**: voci duplicate o incongruenze del database

Perché sono un problema? 1. **Inflazione dell’autorità**: In PageRank, i self-loop creano cicli di feedback dove i nodi aumentano artificialmente il proprio

punteggio 2. **Genealogia fuorviante:** L'autocampionamento non rappresenta l'influenza tra artisti diversi

Esempio concreto – “Ninja McTits”: Un artista (“Ninja McTits”) aveva 443 self-loop. Dopo aver calcolato PageRank, questo artista otteneva un punteggio di 66,45 – drammaticamente più alto del legittimo primo in classifica (Daniel Ingram: 60,05). Rimuovendo i self-loop, Ninja McTits scompare completamente dalla classifica.

Soluzione:

```
df_graph = df_graph.filter(  
    col("Sampler_Artist_Name") != col("Original_Artist_Name")  
)
```

Dopo la rimozione, gli archi si sono ridotti da 22.135 a 20.728 a livello di artista.

Nota importante: Il grafo finale a livello di canzone contiene 57.741 eventi di sampling (relazioni recording-to-recording), che vengono poi aggregati in archi pesati a livello di artista per le analisi PageRank e clustering. I 57.741 riflettono interazioni fine-grained tra canzoni, mentre i 20.728 rappresentano la topologia artista-a-artista usata per l'analisi strutturale.

3.7 Risoluzione degli alias

Un altro problema di qualità dei dati: lo stesso artista può apparire con nomi diversi nel database.

Esempi: - “James Brown” vs “James Brown & The Famous Flames” - “The Beatles” vs “Beatles, The” - “Queen” vs “Queen (Band)”

Ho creato una mappa di alias con 161 varianti di nomi d'artista, unendo automaticamente le varianti. Ad esempio, tutte le occorrenze di “James Brown & The Famous Flames” vengono mappate a “James Brown”.

3.8 Ottimizzazione: formato Parquet

Il costo computazionale per costruire il grafo (multiple join su milioni di righe) è significativo. Per evitare di ricalcolare tutto a ogni analisi, il grafo finale è stato salvato su disco in formato **Apache Parquet**.

Vantaggi del Parquet vs CSV: 1. **Archiviazione colonnare:** Legge solo le colonne necessarie per una specifica query (es. solo `source_id` e `target_id`), riducendo l'I/O 2. **Preservazione dello schema:** Mantiene i metadati e i tipi di dato (interi, stringhe), eliminando la necessità di inferire lo schema a ogni lettura 3. **Compressione:** Riduce lo spazio su disco (algoritmi efficienti come Snappy e Gzip)

4. Metodologia e Algoritmi – Sezione 3 del Report

Con il grafo strutturato e ottimizzato, l'analisi si è concentrata sull'estrarre conoscenza attraverso tre livelli di complessità: **statistiche descrittive**, **analisi dell'autorità strutturale** (PageRank) e **rilevamento delle comunità** (clustering).

Punto cruciale: Per sfruttare appieno la natura distribuita di Spark e dimostrare una comprensione profonda della teoria dei grafi, ho implementato PageRank **da zero** usando trasformazioni PySpark (paradigma MapReduce), invece di usare librerie pre-costruite come GraphFrames.

4.1 Analisi descrittiva: Degree Centrality

Il primo livello di analisi quantifica il “volume” usando metriche base di topologia dei grafi.

In-Degree (Più campionati) L'**in-degree** conta il numero di archi entranti per un nodo. In questo contesto, quantifica la popolarità grezza di un artista come fonte di materiale.

Esempio: Se James Brown è stato campionato da 203 artisti diversi, il suo in-degree è 203.

```
-- Semplificato: conto quante volte ogni artista viene campionato
SELECT Original_Artist_Name, COUNT(*) AS in_degree
FROM grafo
GROUP BY Original_Artist_Name
ORDER BY in_degree DESC
```

Out-Degree (Più campionatori) L'**out-degree** conta il numero di archi uscenti. Identifica gli artisti che fanno più affidamento sul sampling per il loro processo creativo.

Esempio: Se un produttore hip-hop usa 50 campioni diversi in un album, il suo out-degree è 50.

Classifica	Per Volume (In-Degree)	Conteggio
1	Daft Punk	370
2	Lady Gaga	281
3	The Beatles	267
4	JAY-Z	260
5	PSY	253

4.2 PageRank: l'autorità strutturale

Il problema che risolve L'**In-Degree** misura la **popolarità** (quantità), ma non cattura la **qualità** dell'influenza. Essere campionati da un artista oscuro

non è la stessa cosa che essere campionati da un artista famoso.

Esempio:

- Artista A viene campionato 10 volte da artisti sconosciuti -> in-degree = 10
- Artista B viene campionato 5 volte da artisti famosissimi -> in-degree = 5

Chi è più influente? Probabilmente l'Artista B, ma il semplice conteggio non lo mostra.

Come funziona PageRank PageRank assegna un punteggio basato sul principio che *“essere campionati da un artista influente trasmette più autorità che essere campionati da uno minore”*.

La formula è:

$$PR(A) = (1 - d) + d \cdot \sum_{B \in \text{incoming}(A)} \frac{PR(B)}{\text{OutDegree}(B)}$$

Dove: - $PR(A)$ = PageRank dell'artista A - d = **damping factor** (0,85) – rappresenta la probabilità che un “navigatore musicale ideale” continui a seguire le relazioni di sampling invece di saltare a un artista a caso - $(1 - d)$ = probabilità di “teleportazione” (saltare a un artista a caso), che garantisce che il grafo sia connesso - $PR(B)$ = PageRank dell'artista B (che campiona A) - $\text{OutDegree}(B)$ = numero di artisti che B campiona

Esempio passo-passo Consideriamo un grafo con 3 artisti:

James Brown <- 2Pac <- Kanye West

- Kanye West campiona 2Pac
- 2Pac campiona James Brown

Passo 1 – Inizializzazione:

Tutti gli artisti iniziano con PageRank = $1/3 \approx 0,333$

Passo 2 – Iterazione 1: - James Brown: riceve autorità da 2Pac - $PR(\text{James}) = (1-0,85) + 0,85 \times PR(2\text{Pac})/\text{OutDegree}(2\text{Pac}) = 0,15 + 0,85 \times 0,333/1 = 0,15 + 0,283 = 0,433$

- **2Pac:** riceve autorità da Kanye West
– $PR(2\text{Pac}) = 0,15 + 0,85 \times 0,333/1 = 0,433$
- **Kanye West:** non riceve da nessuno (è il “sampler” puro)
– $PR(\text{Kanye}) = 0,15 + 0,85 \times 0 = 0,15$

Passo 3 – Iterazione 2: - James Brown: $0,15 + 0,85 \times 0,433/1 = 0,518$ - **2Pac:** $0,15 + 0,85 \times 0,15/1 = 0,277$ - **Kanye West:** $0,15 + 0,85 \times 0,433/\dots$ aspetta, Kanye non riceve archi, ma abbiamo i dangling nodes!

Il problema dei Dangling Nodes (Nodi Sorgente) Nel nostro esempio, **Kanye West** campiona 2Pac ma non è campionato da nessuno: non ha archi entranti. Ma dal punto di vista dell'algoritmo, il problema è l'opposto: **Kanye West non ha archi uscenti** verso altri sampler (è un "sink" / pozzetto).

Nella nostra implementazione, gli archi vanno dal **sampler all'originale** (Kanye -> 2Pac -> James Brown). Quindi l'autorità scorre in **direzione opposta** agli archi. I dangling nodes sono i nodi che **non campionano nessuno** (es. James Brown, se non campionasse altri).

Per gestirli, l'algoritmo distribuisce uniformemente il loro punteggio a tutti gli altri nodi.

Implementazione in PySpark

```
# Pseudo-codice dell'implementazione PageRank

# 1. Inizializzazione: tutti i nodi con rank = 1/N
ranks = nodes.map(lambda n: (n, 1.0 / N))

# 2. Per 50 iterazioni:
for i in range(50):
    # Contributi: ogni nodo divide il suo rank per i suoi out-degree
    contribs = graph.join(ranks).flatMap(
        lambda (node, (outlinks, rank)):
            [(dest, rank / len(outlinks)) for dest in outlinks]
    )

    # Aggregazione: sommo i contributi per ogni nodo
    ranks = contribs.reduceByKey(lambda a, b: a + b)

    # Applico la formula di PageRank
    ranks = ranks.mapValues(
        lambda rank: 0.15 + 0.85 * rank
    )

# Gestione dangling nodes
# (se un nodo non ha out-degree, distribuisco il suo rank a TUTTI)
```

Sfide tecniche affrontate

1. **Dangling Nodes:** Usando `right_outer_join` per identificare i nodi senza archi uscenti e ridistribuire il loro punteggio
2. **Lineage Truncation:** Spark costruisce un "albero genealogico" delle trasformazioni (lineage) che cresce a ogni iterazione. Dopo 50 iterazioni, questo può causare uno `StackOverflowError`. La soluzione è usare `.checkpoint()` a ogni iterazione per salvare lo stato intermedio su disco e

troncare il lineage.

4.3 Community Detection: Algoritmo di Louvain

Cos'è una “comunità” in un grafo? Una comunità è un gruppo di nodi densamente connessi tra loro, con poche connessioni verso l'esterno. In termini musicali, una comunità rappresenta una “famiglia genealogica” – artisti che condividono le stesse fonti di sampling.

Come funziona Louvain L'algoritmo di Louvain è un algoritmo gerarchico che massimizza la **modularità** – una misura di quanto il grafo è ben diviso in comunità. Funziona in due fasi che si ripetono:

Fase 1 – Ottimizzazione locale: 1. Per ogni nodo, valuta quanto aumenterebbe la modularità spostandolo nella comunità di ciascun vicino 2. Sposta il nodo nella comunità che dà l'aumento maggiore 3. Ripete finché non si può più migliorare

Fase 2 – Aggregazione: 1. Le comunità trovate diventano “super-nodi” 2. Viene costruito un nuovo grafo dove i pesi rappresentano le connessioni tra comunità 3. La Fase 1 si ripete su questo nuovo grafo

Esempio concreto:

Immaginiamo 6 artisti hip-hop che campionano tutti James Brown, e 4 artisti rock che campionano tutti i Led Zeppelin. L'algoritmo creerà due comunità separate: - Comunità 1: James Brown + 6 artisti hip-hop - Comunità 2: Led Zeppelin + 4 artisti rock

Nonostante l'algoritmo non sappia cosa siano “hip-hop” o “rock”, le relazioni strutturali rivelano queste famiglie musicali.

Dettagli implementativi

- **Grafo pesato:** Ogni arco diretto (A campiona B) diventa un arco indiretto con peso = 1 (o il numero di eventi di sampling tra la coppia)
- **Risoluzione $r = 1.0$:** Controlla quanto l'algoritmo è “fine” nel trovare comunità. Valori più alti creano più comunità piccole, valori più bassi creano meno comunità grandi
- **Implementazione:** NetworkX `community.louvain_communities`

Perché è unsupervised (non supervisionato) Un punto cruciale: l'algoritmo opera **senza sapere nulla dei generi musicali**. Non abbiamo importato etichette come “Hip Hop” o “Rock”. L'algoritmo si basa esclusivamente su **pattern strutturali**: se un insieme di artisti campiona sistematicamente le stesse fonti, vengono raggruppati nello stesso cluster.

Questo verifica l'ipotesi che **i generi musicali sono fondamentalmente comunità di pratica condivisa**.

5. Risultati – Sezione 4 del Report

5.1 Statistiche di rete

Il grafo finale processato, dopo il data engineering e la rimozione dei self-loop, comprende:

Metrica	Valore	Interpretazione
Densità del grafo	0,000013	Estremamente sparso, tipico delle reti di influenza reali
In-Degree Medio	3,50	In media un artista viene campionato 3,5 volte
Out-Degree Medio	5,95	In media un artista usa 5,95 campioni
In-Degree Massimo	370	Daft Punk è l'artista più campionato

5.2 Distribuzione Power-Law (Legge di potenza)

La rete mostra caratteristiche **scale-free** (senza scala), tipiche delle reti ad attaccamento preferenziale: - Il **top 1%** degli artisti controlla il **23,5%** di tutti gli eventi di sampling - Il **top 5%** controlla il **46,5%** - Il **top 10%** controlla il **58,3%**

Cosa significa? Pochissimi artisti (le “superstar”) accumulano la stragrande maggioranza dei campionamenti, mentre la stragrande maggioranza degli artisti viene campionata pochissimo. Questo stesso pattern si osserva nelle reti di citazioni accademiche, nei social network e nel World Wide Web.

Esempio visivo: Se ordiniamo gli artisti per numero di volte che sono stati campionati (in-degree), la classifica sarebbe qualcosa del tipo:

Daft Punk:	#####	370
Lady Gaga:	#####	281
The Beatles:	#####	267
JAY-Z:	#####	260
...		
Artista #1000:		2
Artista #5000:		1
Artista #10000:		0

5.3 Classifiche PageRank

Dopo 50 iterazioni (tolleranza $< 0,01$), PageRank produce punteggi di autorità strutturale.

Top 10 per Volume vs Autorità

#	Per Volume (In-Degree)	Eventi	Per Autorità (PageRank)	Punteggio
1	Daft Punk	370	Daniel Ingram	59,85
2	Lady Gaga	281	James Brown	45,67
3	The Beatles	267	2Pac	45,01
4	JAY-Z	260	Denonbu (Denonbu)	40,90
5	PSY	253	Gaiten Bungebun High School	35,61
6	Linkin Park	230	Lyn Collins	27,10
7	Michael Jackson	220	Daft Punk	27,08
8	Beastie Boys	209	Kaihatsu Pixle (Kaihatsu)	26,98
9	James Brown	203	John Lennon	26,06
10	Katy Perry	186	Kraftwerk	25,99

5.4 Volume vs Autorità: gli influencer nascosti

Questa sezione confronta i risultati della Degree Centrality (Volume) con PageRank (Autorità).

I Re del Volume Artisti pop ed elettronici moderni come **Daft Punk** (370 eventi) e **Lady Gaga** (282) dominano le metriche di volume. Sono gli artisti più campionati in termini assoluti.

Le Autorità Classiche **James Brown**, **Lyn Collins** e **Kraftwerk** hanno punteggi PageRank alti nonostante volumi raw più bassi. Questo significa che **chi li campiona è a sua volta molto influente** – un segno distintivo di vera importanza strutturale.

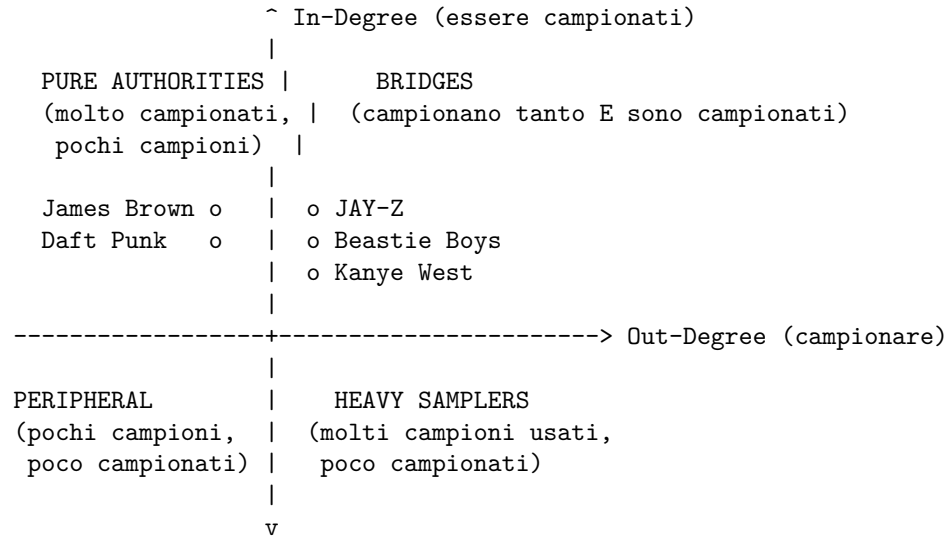
Il caso Daniel Ingram **Daniel Ingram** (compositore per *My Little Pony*) raggiunge il PageRank più alto (59,85) nonostante solo 130 eventi di sampling. Come vedremo nell’analisi del contesto di autorità, questo è guidato da una comunità di fan densa e isolata, non da un’influenza mainstream.

La sorpresa giapponese **Denonbu (Denonbu)** e **Gaiten Bungebun High School** sono progetti multimediali giapponesi che emergono come autorità inaspettate. La musica elettronica giapponese e le colonne sonore di videogiochi/anime hanno un’enorme influenza strutturale nel campionamento moderno, specialmente nella scena delle “chip tune” e del “future funk”.

5.5 Classificazione degli artisti: Hub Analysis & Bridges

Il report classifica gli artisti in quattro categorie basate sul loro comportamento di sampling:

I quattro quadranti



1. **Pure Authorities** (alto in-degree, basso out-degree): Artisti come James Brown e Daft Punk – campionati estensivamente ma usano pochi campioni loro stessi. Sono il “materiale sorgente” della musica moderna.
2. **Bridges** (alto in-degree, alto out-degree): Artisti che **sia campionano tanto CHE vengono campionati**. Nodi evolutivi come **JAY-Z, Beastie Boys** e **Kanye West** mantengono un equilibrio quasi perfetto, agendo come il tessuto connettivo cruciale della musica moderna.
3. **Heavy Samplers** (basso in-degree, alto out-degree): Produttori moderni che fanno molto affidamento sul sampling ma non sono ancora diventati autorità essi stessi.
4. **Peripheral Artists** (basso in-degree, basso out-degree): Artisti con metriche sotto la mediana. Rappresentano la maggior parte dei nodi nella “lunga coda” della distribuzione.

5.6 Validazione esterna contro WhoSampled

Per verificare la robustezza delle classifiche PageRank, abbiamo confrontato i nostri risultati con una lista di verità (ground truth) da **WhoSampled** (un sito indipendente di riferimento per il sampling).

Metodo

1. Abbiamo preso la lista dei 10 artisti più campionati secondo WhoSampled
2. Abbiamo confrontato il loro ordinamento con il nostro PageRank
3. Abbiamo calcolato la **correlazione di Spearman** (una misura di correlazione basata sui ranghi, non sui valori assoluti)

Risultato

- **Correlazione:** $\rho = 0,72$ ($p = 0,29$, $n = 10$)
- **Interpretazione:** Correlazione positiva moderata, ma non statisticamente significativa a causa del campione piccolo ($p > 0,05$)

Cosa significa $\rho = 0,72$?

La correlazione di Spearman varia da -1 (perfetta correlazione inversa) a +1 (perfetta correlazione diretta).

- 0 = nessuna correlazione - 0,72 = forte correlazione positiva

Significa che, in generale, gli artisti che WhoSampled considera più campionati tendono ad avere punteggi PageRank alti. Ma non è una corrispondenza perfetta, perché PageRank cattura una dimensione diversa (l'influenza strutturale, non solo il volume).

Osservazione chiave: PageRank promuove artisti strutturalmente influenti come **Lyn Collins** (WhoSampled rank 5 -> sistema rank 2) e **Kraftwerk** (WhoSampled rank 6 -> sistema rank 3), rispetto ad artisti ad alto volume ma meno centrali strutturalmente.

5.7 Contesto di Autorità: influenza interna vs esterna

Il problema PageRank modella efficacemente l'autorità, ma non distingue naturalmente tra: - **Influenza mainstream globale:** un artista è campionato da una comunità grande e diversificata - **Dominanza di comunità isolata:** un artista è campionato da una piccola comunità densa

Il caso Daniel Ingram Daniel Ingram ha il PageRank più alto (59,85) ma solo 130 eventi di sampling. Per investigare questa "anomalia", abbiamo analizzato la composizione dei campioni in entrata per i top 15 artisti per autorità.

Risultato: - **Daniel Ingram:** l'89% dei suoi campioni proviene dalla **sua stessa comunità** (circa 50-100 fan remixer di *My Little Pony*) - **James Brown:** il 74% dei suoi campioni proviene dalla **sua comunità**, ma questa comunità è enorme (2.196 artisti dell'ecosistema hip-hop/funk/soul)

Artista	% Campioni Interni	Dimensione Comunità	Interpretazione
Daniel Ingram	89%	~50-100	Autorità “gonfiata” da comunità piccola e densa
James Brown	74%	2.196	Autorità globale legittima

L’Ipotesi dell’Amplificazione tramite Densità di Comunità:

PageRank amplifica l’autorità all’interno di cluster densi e altamente interconnessi, indipendentemente dalla loro dimensione assoluta. Quindi, mappare il contesto della comunità è essenziale per determinare se l’autorità di un artista è veramente globale o solo un artefatto di una micro-comunità isolata.

Conseguenza: Daniel Ingram è il “re” della sua piccola comunità, non un’influencer globale. James Brown, al contrario, è il re di un vasto impero musicale.

5.8 Famiglie genealogiche: rete colorata per cluster

Il report visualizza i top 50 artisti per PageRank, con ogni nodo colorato in base al suo cluster Louvain: - **Archi solidi:** sampling intra-cluster (stessa comunità) - **Archi tratteggiati:** sampling inter-cluster (comunità diverse) - **Dimensione del nodo:** proporzionale al PageRank (autorità)

5.9 Rilevamento delle comunità

L’algoritmo Louvain ha rilevato **1.774 comunità distinte** con una frazione di archi intra-cluster di **0,7169**.

Cosa significa 0,7169?

Circa il **72%** delle connessioni di sampling avviene tra artisti nella stessa famiglia genealogica, mentre solo il **28%** attraversa i confini delle comunità.

La comunità più grande: Contiene **2.196 artisti** ed è centrata intorno a **JAY-Z**. Rappresenta il nucleo dominante e altamente integrato del sampling moderno (hip-hop, R&B, funk, soul).

Perché 1.774 e non di più? Questo numero relativamente contenuto di comunità riflette la capacità dell’algoritmo Louvain di trovare partizioni gerarchiche ottimali senza frammentare eccessivamente la rete.

Esempio di come interpretare una comunità:

Se la Comunità #1 contiene James Brown, Lyn Collins, 2Pac, e migliaia di artisti hip-hop che li campionano, possiamo chiamarla la “comunità funk/hip-hop”. Non perché le abbiamo dato noi questa etichetta, ma perché la struttura

del grafo rivela che questi artisti condividono le stesse fonti e sono densamente interconnessi.

6. Discussione e Interpretazione – Sezione 5 del Report

6.1 Implicazioni teoriche

Questa analisi dimostra che l'influenza musicale è fondamentalmente un **fenomeno di rete** governato da proprietà strutturali, non solo da metriche di popolarità.

Tre scoperte chiave:

1. **Autorità vs Popolarità:** PageRank rivela che la **posizione strutturale** (essere campionati da artisti influenti) conta tanto quanto il volume grezzo. Questo spiega perché certe franchise native di internet (anime, videogiochi, My Little Pony) possono superare in classifica artisti legacy con più campioni totali.
2. **Genealogia cross-genere:** L'emergere di franchise multimediali (animazione, videogiochi) come massime autorità dimostra che il sampling crea percorsi genealogici inaspettati che vanno oltre i confini tradizionali dei generi musicali.
3. **Attaccamento preferenziale:** La distribuzione power-law conferma che l'influenza musicale segue gli stessi pattern matematici delle reti di citazioni, dei social network e del World Wide Web – pochi hub dominano mentre la maggior parte dei nodi rimane periferica.

6.2 Contributi metodologici

1. **Implementazione da zero:** Implementare PageRank manualmente in PySpark dimostra una comprensione profonda degli algoritmi distribuiti su grafi.
2. **Data Engineering:** Aver navigato con successo lo schema relazionale complesso di MusicBrainz (Terza Forma Normale) e trasformato in una struttura a grafo dimostra competenze reali di data engineering.
3. **Ottimizzazione della convergenza:** Lo schema a 50 iterazioni fisse con monitoraggio della convergenza e corretta redistribuzione della massa dei nodi sink garantisce punteggi PageRank stabili.
4. **Strategia di visualizzazione:** La creazione di visualizzazioni pulite e non sovrapposte rende interpretabili strutture complesse.

6.3 Limiti e lavori futuri

Limiti identificati

1. **Qualità dei dati e risoluzione entità:** Nonostante la rimozione dei self-loop, il database MusicBrainz contiene ancora incongruenze come artisti placeholder e voci duplicate.
2. **Bias di campionamento e attualità dati:** MusicBrainz si basa su metadati contribuiti dagli utenti, che potrebbero sottorappresentare certe regioni geografiche o generi non occidentali. Inoltre, il dataset rappresenta una fotografia statica; incorporare le date di pubblicazione permetterebbe analisi temporali.
3. **Limiti di scala:** PageRank è stato eseguito per 50 iterazioni. Estendere l'analisi a PageRank dinamico o personalizzato potrebbe dare ulteriori approfondimenti.
4. **Validazione dei generi:** L'algoritmo Louvain è unsupervised e raggruppa i nodi in base alla prossimità strutturale. Incorporare etichette di genere esplicite permetterebbe di validare formalmente se questi cluster strutturali si allineano con le definizioni convenzionali di genere.
5. **Genealogia multi-hop:** Estendere l'analisi a catene di sampling multi-generazionali (Originale -> Campione 1 -> Ricampionamento) rivelerebbe percorsi genealogici più profondi.

Lavori futuri

- **Rilevamento automatico di outlier:** Implementare algoritmi per identificare automaticamente voci anomale nei dati
- **Analisi temporale:** Studiare come i pattern di sampling si evolvono attraverso i decenni
- **PageRank personalizzato:** Calcolare l'autorità relativa a specifici generi o epoche
- **Validazione con generi musicali:** Usare i tag di genere di MusicBrainz per validare formalmente i cluster rilevati

7. Conclusione – Sezione 6 del Report

Riassunto delle scoperte chiave

1. **Nuove autorità inaspettate:** **Daniel Ingram** (compositore per *My Little Pony*) e progetti elettronici giapponesi come **Denonbu (Denonbu)** emergono come le nuove autorità strutturali, dimostrando l'enorme influenza delle franchise multimediali moderne e della cultura internet sul campionamento contemporaneo.

2. **I classici rimangono fondamentali:** **James Brown** e le icone funk classiche rimangono pilastri fondamentali, ma ora condividono il gradino più alto con creatori dell'era digitale.
3. **Distribuzione power-law:** Il top 1% degli artisti controlla il 23,5% di tutti gli eventi di sampling, confermando la natura scale-free della rete.
4. **Forte struttura comunitaria:** 1.774 famiglie genealogiche rilevate, con il 72% del sampling che rimane all'interno dei confini comunitari. Il panorama musicale moderno è fortemente compartimentato.
5. **Convergenza PageRank:** Raggiunta con corretta redistribuzione della massa dei sink in 50 iterazioni.
6. **Validazione:** Correlazione di Spearman di 0,72 contro WhoSampled conferma che PageRank cattura dimensioni dell'influenza strutturale.

Il messaggio finale

Questa analisi sposta la conversazione da “chi ha venduto più dischi” a “chi ha strutturalmente plasmato la musica moderna”. Sfruttando la teoria dei grafi e il calcolo distribuito, abbiamo mappato il DNA del suono contemporaneo – rivelando che l'influenza scorre attraverso le reti, non le classifiche.

Impatto e applicazioni future

Questa metodologia potrebbe estendersi ad altri domini: - **Reti di citazioni accademiche:** Quali articoli sono fondamentali? - **Influenza nei social media:** Chi guida veramente la conversazione? - **Dipendenze software:** Quali librerie sostengono lo sviluppo moderno?

8. Struttura del Progetto

```
DataMiningCorrao/
|
+-- README.md           <- Documentazione principale del progetto
+-- .gitignore           <- File di esclusione Git
+-- .ruff_cache/         <- Cache del linter Python
+-- .venv/               <- Ambiente virtuale Python (dipendenze pre-installate)
+-- SPIEGAZIONE_PROGETTO.md <- QUESTO FILE - spiegazione completa in italiano
|
+-- project/             <- TUTTO il codice sorgente e i dati
|   +-- README.md        <- Documentazione tecnica
|   +-- requirements.txt  <- Dipendenze Python
|   +-- run_pipeline.sh   <- Pipeline automatica in 12 passi
|   |
```

```

| +--- src/                                <- Script di analisi principali
| | +--- data_preparation.py              <- Ingestione dati + costruzione grafo (231 righe)
| | +--- top_ranking.py                   <- Analisi degree centrality (46 righe)
| | +--- compute_authority_manual.py      <- PageRank implementato da zero (123 righe)
| | +--- cluster.py                       <- Louvain community detection (97 righe)
| | +--- validation_metrics.py            <- Statistiche di rete (231 righe)
| | +--- cluster_quality.py               <- Qualità dei cluster (242 righe)
| | +--- external_validation.py           <- Validazione WhoSampled (136 righe)
| | +--- genealogy_visualizations.py      <- Figura 2: Hub + Bridges (200 righe)
| | +--- advanced_experiments.py          <- Figura 3: Contesto autorità (157 righe)
| | +--- generate_report_figures.py       <- Figure 1 + 5 e statistiche (351 righe)
|
| +--- utils/                             <- Script di utilità
| | +--- analyze_selfloops.py             <- Analisi self-loop (140 righe)
| | +--- visualize_cluster.py             <- Figura 4: Rete colorata per cluster (222 righe)
| | +--- generate_interactive_network.py   <- Visualizzazione D3.js (509 righe)
|
| +--- data/                              <- Dati di alias
| | +--- build_alias_map.py               <- Costruttore mappa alias (115 righe)
| | +--- artist_aliases.json              <- 161 alias manuali
| | +--- alias_auto.json                  <- Alias generati automaticamente
|
| +--- mbdump/                            <- Dati grezzi MusicBrainz (input TSV)
| | +--- artist                          <- Tabella artisti
| | +--- artist_credit_name               <- Crediti artisti
| | +--- recording                       <- Registrazioni (canzoni)
| | +--- l_recording_recording            <- Tabella ponte (sampling)
| | +--- link                            <- Metadati link
| | +--- track                           <- Tracce
| | +--- medium                           <- Supporti
| | +--- release                          <- Pubblicazioni
| | +--- release_group_meta               <- Metadati gruppo pubblicazione
|
| +--- outputs/                           <- Risultati generati
| | +--- music_graph.parquet/             <- Grafo pulito (~15MB)
| | +--- artist_pagerank.parquet/         <- Punteggi PageRank (~2MB)
| | +--- music_labels.parquet             <- Assegnazioni comunità
| | +--- music_clusters.csv               <- Statistiche cluster
| | +--- music_cluster_membership.csv      <- Mapping artista -> cluster
| | +--- validation_summary.csv/          <- Statistiche di rete
| | +--- cluster_quality_summary.csv/     <- Metriche qualità cluster
| | +--- top_100_artists_pagerank.csv/    <- Top 100 per autorità
| | +--- cluster_sizes.csv/               <- Distribuzione dimensioni cluster
| | +--- cluster_bridges.csv/             <- Connessioni inter-cluster
| | +--- external_validation.csv          <- Confronto artisti matched
| | +--- external_validation_meta.txt      <- Risultato Spearman (rho=0,72)

```

```

| |   +--- interactive_genealogy.html <- Visualizzazione D3.js (~3MB)
| |
| |   +--- figures/report_figures/      <- Figure del report (PNG + PDF)
| |   +--- fig1_volume_vs_authority.* <- Confronto Volume vs Autorità
| |   +--- fig2_hub_bridges.*          <- Hub analysis + Bridges
| |   +--- fig3_authority_context.*    <- Contesto autorità
| |   +--- fig4_cluster_artist_network.* <- Rete colorata per cluster
| |   +--- fig5_cluster_distribution.* <- Distribuzione cluster
| |   +--- statistics_summary.txt      <- Riepilogo statistiche
| |
| |   +--- checkpoints/                <- Checkpoint Spark PageRank
| |   +--- checkpoints_clustering/     <- Checkpoint Spark clustering
| |
| +--- report/                        <- Report finale
|   +--- DataMining.tex               <- Sorgente LaTeX (572 righe)
|   +--- DataMining.pdf               <- PDF compilato (consegna principale)
|   +--- Tesi.bib                     <- Bibliografia (6 riferimenti)
|   +--- Immagini/                   <- Figure e immagini
|     +--- fig1_volume_vs_authority.pdf/png
|     +--- fig2_hub_bridges.pdf/png
|     +--- fig3_authority_context.pdf/png
|     +--- fig4_cluster_artist_network.pdf/png
|     +--- fig5_cluster_distribution.pdf/png
|     +--- music_brainz_download_db.png
|     +--- who_sampled.png
|     +--- sigillo_unitn.png
|     +--- statistics_summary.txt
|     +--- table_top10_comparison.tex

```

Stack tecnologico

Tecnologia	Versione	Ruolo
Apache Spark (PySpark)	3.5.0	Elaborazione distribuita, PageRank custom
NetworkX	3.x	Louvain community detection, analisi grafi
Matplotlib / Seaborn	3.x	Figure pubblicazione (300 DPI)
D3.js	-	Visualizzazione interattiva web

Tecnologia	Versione	Ruolo
FuzzyWuzzy	0.18.0	Fuzzy matching per validazione esterna
LaTeX (pdflatex)	-	Compilazione report con minted
Parquet	-	Storage colonnare per I/O efficiente

*Questo file è stato creato come spiegazione completa e dettagliata del progetto
“The Genealogy of Sound: Music Sampling Network Analysis” per il corso di
Data Mining 2025/2026 dell’Università di Trento.*